

Capstone Checkpoint 2.1: Capstone Agent Design Refinement

Author: Jacques Troussard

1. Reasoning Loop Strategy and Breakdown

The main agent will use an adaptive ReAct-style reasoning loop. Its overall goal is to transform an unstructured document into a validated, searchable, human-navigable record.

The general workflow is:

Document → Understanding → Structured Document Record → Validation → Organization → Filesystem

At each stage, the agent reasons about the current state, selects an appropriate action, observes the result, and uses that observation to determine its next step:

Reason → Act → Observe → Reason Again

The agent will not necessarily use the same action or reasoning process at every stage. Straightforward tasks may require only a single action, while ambiguous results may require additional reasoning or verification. A separate set of organizational rules will provide guidance when the agent needs to determine how a document should be organized within the final directory structure location.

For example, if the agent cannot obtain sufficient text from a scanned document, it may determine that OCR is necessary. The OCR result becomes a new observation, allowing the agent to reassess its understanding and continue processing. If the agent still cannot resolve an ambiguity, it can defer to human review.

2. Memory Requirements

The agent will require both short-term and long-term memory.

Short-term memory is needed to maintain the context of the document currently being processed. This includes extracted text, OCR results, metadata being generated, observations from tools, and decisions made during the current processing cycle.

Long-term memory is needed to maintain information across documents and processing sessions. Examples include previously identified entities, organizational rules, document patterns, and corrections made by the user during manual review.

Long-term memory is particularly important because the meaning of a new document may depend on information learned from earlier documents. For example, once the system has identified a particular vehicle or organization or document style, later documents referring to it can be interpreted using that existing information.

3. External Tools

The agent will use external tools when tasks require capabilities that an LLM does not inherently possess or the generated results in low confidence.

One necessary tool is OCR. A language model may be able to interpret text, but a scanned image may not contain usable text at all. OCR provides the missing textual representation that allows the agent to reason about the document.

Other tools may include filesystem operations, document parsing, hashing for duplicate detection, and a local database for retrieving previously stored information.

These tools allow the agent to interact with its environment rather than relying entirely on information contained in a prompt.

4. Improvement Over a Prompt-Only Approach

A single prompt would be insufficient because the document processing task requires interaction with external data, persistent information, and the results of previous actions.

For example, a prompt-only system might receive a scanned document and attempt to classify it without being able to read the image accurately. It could also generate a reasonable directory path without knowing that the same entity has another path already defined and loaded with documents.

The agentic design addresses these failures by allowing the system to take actions, observe their results, retain relevant information, and reconsider its decision when new evidence becomes available.

This makes the system more reliable than simply asking an LLM to classify and organize each document in isolation.

5. Initial Design Scope

This design represents the initial hypothesis for how the agent could solve the problem rather than a final implementation specification. The core goal is to establish a reasoning process that can interact with tools, maintain relevant memory, respond to new observations, and involve a human when it cannot confidently resolve a document.

The design will continue to evolve as the agent is implemented and tested. Later development may identify opportunities to divide responsibilities between additional components or use different reasoning strategies for different parts of the workflow.